Lab 14 – Implementation of Global Data Flow Analysis

Aim

To implement global data flow analysis using reaching definitions.

Theory

A definition of a variable **reaches** a program point if there exists a path from that definition to the point without any redefinition of that variable.

For each basic block:

- **IN[B]** = set of definitions reaching the beginning of block B
- **OUT[B]** = set of definitions reaching the end of block B

Formula:

OUT[B] = GEN[B] ∪ (IN[B] − KILL[B])

Algorithm

1. Start the program.
2. Read the number of basic blocks.
3. For each block, read **GEN** and **KILL** sets.
4. Initialize **IN** of entry block as empty.
5. Compute **OUT** using
   OUT = GEN ∪ (IN − KILL).
6. For subsequent blocks:
   - IN is previous block's OUT.
7. Repeat until no changes occur.
8. Display **IN** and **OUT** sets.
9. Stop.

C Program

```
#include <stdio.h>

int main() {
    int n;
    int gen[10], kill[10];
    int in[10], out[10];

    printf("Enter number of basic blocks: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
```

```
        printf("Enter GEN and KILL for block %d: ", i + 1);
        scanf("%d %d", &gen[i], &kill[i]);
    }
    in[0] = 0;
    out[0] = gen[0];

    for (int i = 1; i < n; i++) {
        in[i] = out[i - 1];

        if (in[i] == kill[i])
            out[i] = gen[i];
        else
            out[i] = gen[i] + in[i];
    }

    printf("\nGlobal Data Flow Analysis\n");
    printf("Block\tGEN\tKILL\tIN\tOUT\n");

    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\n",
               i + 1,
               gen[i],
               kill[i],
               in[i],
               out[i]);
    }

    return 0;
}
```

Sample Input

Enter number of basic blocks: 3
Enter GEN and KILL for block 1: 1 0
Enter GEN and KILL for block 2: 2 1
Enter GEN and KILL for block 3: 3 2

Sample Output

Global Data Flow Analysis

Block	GEN	KILL	IN	OUT
1	1	0	0	1
2	2	1	1	2
3	3	2	2	3

Result

Thus the global data flow analysis using reaching definitions was implemented successfully.